Solving the Maximum Cut Problem using GRASP

CSE 318: Artificial Intelligence Sessional — Assignment 2

K M Fahim Hossain
Student ID: 2205090

July 2026

Overview

This report summarizes the implementation and experimental evaluation of five algorithms for the Maximum Cut (MAX-CUT) problem: a randomized construction heuristic, a greedy construction heuristic, a semi-greedy (GRASP-style) construction heuristic, a local search procedure, and the full GRASP metaheuristic that combines semi-greedy construction with local search over multiple iterations. All five algorithms were implemented and evaluated on the full benchmark set of 54 graphs, of which 24 have a known best solution or upper bound available for comparison.

1 Algorithm Descriptions

1.1 Randomized-1

Each vertex is independently assigned to partition X or Y with probability $\frac{1}{2}$, with no regard to edge weights. Since the resulting cut is highly variable, the procedure is repeated n times and the cut weights are averaged to obtain a stable estimate of the algorithm's expected performance. This serves as the naive baseline against which the remaining algorithms are compared.

1.2 Simple Greedy

The two heaviest-connected vertices (the endpoints of the maximum-weight edge) are used to seed partitions X and Y. Each remaining vertex is then assigned, one at a time, to whichever partition maximizes its immediate contribution to the cut, computed as $\sigma_X(v)$ versus $\sigma_Y(v)$. This is entirely deterministic and produces a single solution per graph.

1.3 Semi-Greedy-1 ($\alpha = 0.5$)

This heuristic follows the same greedy assignment logic but introduces controlled randomness. At each step, the greedy function value $\max\{\sigma_X(v), \sigma_Y(v)\}$ is computed for every unassigned vertex, and a cutoff

$$\mu = w_{\min} + \alpha \cdot (w_{\max} - w_{\min})$$

is used to build a Restricted Candidate List (RCL) containing all vertices whose greedy value is at least μ. A vertex is then chosen uniformly at random from the RCL. With $\alpha = 0.5$, the RCL sits halfway between purely greedy ($\alpha = 0$) and purely random ($\alpha = 1$) selection, giving each construction a different, reasonably good starting solution – exactly the diversification GRASP needs.

1.4 Local Search

Starting from a feasible partition $(S, \bar{S})$, local search repeatedly evaluates the gain $\delta(v)$ of moving each vertex to the opposite side:

$$\delta(v) = \begin{cases} \sigma_S(v) - \sigma_{\bar{S}}(v), & v \in S \\ \sigma_{\bar{S}}(v) - \sigma_S(v), & v \in \bar{S} \end{cases}$$

The best-improving move is applied, and the process repeats until no vertex move improves the cut weight, at which point a local optimum has been reached. In this implementation, local search is applied on top of a randomized (Simple Randomized) starting solution rather than a semi-greedy one, so it explores improvement purely from an unbiased starting point; GRASP, described next, still pairs local search with semi-greedy construction internally.

1.5 GRASP

GRASP combines semi-greedy construction and local search inside an iterative loop: each iteration builds a new randomized solution and improves it with local search, retaining the best solution found so far across all iterations. Because construction is randomized, different iterations explore different regions of the solution space, and local search ensures every candidate reported is at least a local optimum. All GRASP runs in this report use 50 iterations.

2 Experimental Setup

All 54 benchmark graphs were used. Randomized-1 was averaged over 50 repetitions, semi-greedy construction used $\alpha = 0.5$, and GRASP was run for 50 iterations per graph, of which the best and average cut weights are both reported. Local search was applied to a randomized starting solution 10 times per graph to obtain its best and average values; because GRASP's internal local search phase instead starts from a semi-greedy solution, the standalone Local Search results are not directly interchangeable with GRASP's internal iterations.

3 Results

Table 1 reports, for every graph, the number of vertices $|V|$ and edges $|E|$, the single-run values of Randomized-1 and Simple Greedy, the Semi-Greedy-1 value, the local-search and GRASP iteration counts and their respective average/best cut weights, and the known best solution or upper bound where available.

Table 1: MAX-CUT results across all 54 benchmark graphs.

Problem			**Constructive algorithm**			**Local search**		**GRASP-1**		**Known best**
Name	$\|V\|$	$\|E\|$	**Rand.-1**	**Greedy**	**Semi-gr.**	**Iters**	**Avg. val.**	**Iters**	**Best val.**	**or bound**
G1	800	19176	9,587.7	10,956	11,161.6	10	11,324.9	50	11,446	12,078
G2	800	19176	9,587.9	11,030	11,194.8	10	11,375.0	50	11,442	12,084
G3	800	19176	9,585.1	11,012	11,167.7	10	11,335.2	50	11,481	12,077
G4	800	19176	9,598.7	11,031	11,174.0	10	11,343.2	50	11,461	–
G5	800	19176	9,591.7	10,990	11,183.7	10	11,357.3	50	11,457	–
G6	800	19176	61.6	1,515	1,661.0	10	1,875.2	50	1,979	–
G7	800	19176	-75.5	1,362	1,514.6	10	1,727.1	50	1,807	–
G8	800	19176	-87.5	1,331	1,501.8	10	1,705.8	50	1,852	–
G9	800	19176	-3.3	1,507	1,538.3	10	1,753.6	50	1,886	–
G10	800	19176	-71.6	1,350	1,506.3	10	1,720.4	50	1,818	–
G11	800	1600	16.1	434	467.8	10	428.2	50	500	627
G12	800	1600	-1.9	418	445.8	10	419.4	50	492	621
G13	800	1600	18.1	432	474.2	10	433.8	50	510	645
G14	800	4694	2,355.4	2,890	2,941.4	10	2,925.7	50	2,985	3,187
G15	800	4661	2,325.5	2,873	2,921.3	10	2,910.4	50	2,972	3,169
G16	800	4672	2,335.5	2,878	2,920.3	10	2,905.0	50	2,975	3,172
G17	800	4667	2,334.4	2,864	2,920.5	10	2,913.0	50	2,972	–
G18	800	4694	37.8	780	789.5	10	839.2	50	892	–
G19	800	4661	-59.6	681	699.6	10	741.0	50	815	–
G20	800	4672	-24.9	698	730.7	10	784.6	50	841	–
G21	800	4667	-31.9	704	710.9	10	762.6	50	858	–
G22	2000	19990	9,957.4	12,346	12,678.6	10	12,799.8	50	13,000	14,123
G23	2000	19990	10,003.9	12,291	12,689.5	10	12,802.0	50	13,012	14,129
G24	2000	19990	9,975.9	12,367	12,699.3	10	12,805.3	50	13,008	14,131
G25	2000	19990	10,006.3	12,408	12,687.5	10	12,784.3	50	12,985	–
G26	2000	19990	9,997.3	12,355	12,678.0	10	12,818.4	50	12,994	–
G27	2000	19990	-29.0	2,316	2,543.3	10	2,804.5	50	2,931	–
G28	2000	19990	-50.2	2,329	2,519.2	10	2,767.7	50	2,880	–
G29	2000	19990	32.5	2,367	2,597.5	10	2,865.7	50	3,017	–
G30	2000	19990	72.2	2,401	2,600.8	10	2,873.9	50	3,048	–
G31	2000	19990	-46.3	2,290	2,545.2	10	2,746.8	50	2,919	–
G32	2000	4000	12.7	1,058	1,142.8	10	1,046.0	50	1,218	1,560
G33	2000	4000	-13.2	1,000	1,143.6	10	1,023.4	50	1,200	1,537
G34	2000	4000	-19.5	988	1,135.8	10	1,023.0	50	1,198	1,541

continued on next page

Table 1 – continued from previous page

Problem			Constructive algorithm			Local search		GRASP-1		Known best
Name	$\|V\|$	$\|E\|$	**Rand.-1**	**Greedy**	**Semi-gr.**	**Iters**	**Avg. val.**	**Iters**	**Best val.**	**or bound**
G35	2000	11778	5,887.1	7,252	7,375.7	10	7,333.7	50	7,481	8,000
G36	2000	11766	5,893.2	7,225	7,355.3	10	7,326.3	50	7,456	7,996
G37	2000	11785	5,887.1	7,264	7,368.8	10	7,346.1	50	7,476	8,009
G38	2000	11779	5,890.5	7,305	7,371.0	10	7,333.8	50	7,470	–
G39	2000	11778	11.5	1,861	1,826.6	10	2,012.5	50	2,124	–
G40	2000	11766	-47.4	1,834	1,864.5	10	1,951.1	50	2,124	–
G41	2000	11785	-14.1	1,876	1,891.5	10	1,979.0	50	2,134	–
G42	2000	11779	66.0	1,989	1,890.8	10	2,047.3	50	2,230	–
G43	1000	9990	4,987.0	6,188	6,325.7	10	6,385.4	50	6,508	7,027
G44	1000	9990	5,007.2	6,157	6,322.1	10	6,375.8	50	6,496	7,022
G45	1000	9990	4,995.1	6,179	6,329.7	10	6,375.9	50	6,491	7,020
G46	1000	9990	4,996.7	6,150	6,323.2	10	6,373.4	50	6,477	–
G47	1000	9990	4,987.4	6,220	6,333.9	10	6,401.6	50	6,501	–
G48	3000	6000	3,003.6	6,000	6,000.0	10	4,961.4	50	6,000	6,000
G49	3000	6000	2,995.9	6,000	6,000.0	10	4,965.6	50	6,000	6,000
G50	3000	6000	3,003.3	5,880	5,860.0	10	4,993.6	50	5,870	5,988
G51	1000	5909	2,949.5	3,626	3,690.0	10	3,677.6	50	3,754	–
G52	1000	5916	2,957.4	3,650	3,695.8	10	3,673.6	50	3,752	–
G53	1000	5914	2,957.1	3,632	3,685.4	10	3,677.0	50	3,749	–
G54	1000	5916	2,964.4	3,633	3,689.9	10	3,664.6	50	3,750	–

4 Plots and Visualizations

4.1 GRASP best vs. average, G1–G10

Figure 1 compares, for the first ten benchmark graphs, the best cut weight found by GRASP against the average cut weight across its 50 iterations. The two values track each other very closely on every instance, with the best solution only marginally ahead of the average – a sign that GRASP's iterations converge to a tight cluster of high-quality local optima rather than a wide spread of outcomes.

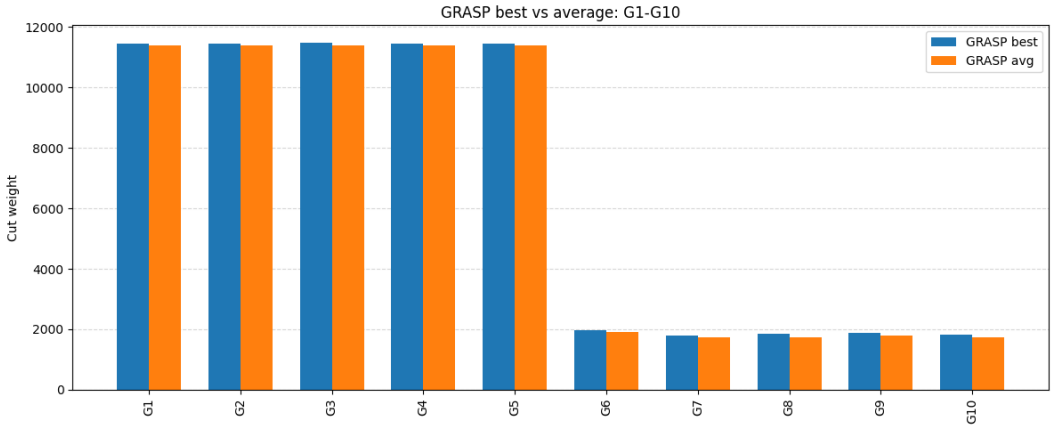

Figure 1: GRASP best vs. average cut weight, graphs G1–G10.

4.2 Algorithm comparison, G1–G10

Figure 2 lays all five algorithms side by side for the same ten graphs. The ordering is consistent across every instance: Randomized $<$ Greedy $<$ Semi-Greedy $<$ Local Search $<$ GRASP, confirming that each added component of the pipeline (greediness, controlled randomization, and then local refinement) contributes a further, cumulative improvement.

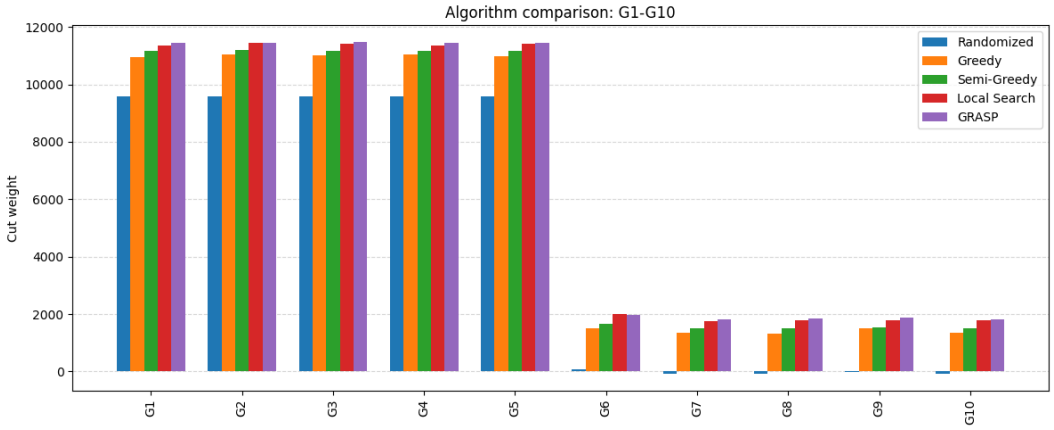

Figure 2: Cut weight comparison across all five algorithms, graphs G1–G10.

4.3 Ratio to known best solution, G1–G3

Figure 3 normalizes results against the known best solution for graphs G1–G3, making the relative gap of each algorithm directly comparable across instances. GRASP and local search both sit consistently around 0.94–0.95 of the known best, while greedy and semi-greedy trail slightly behind, and this gap is stable across the three graphs shown.

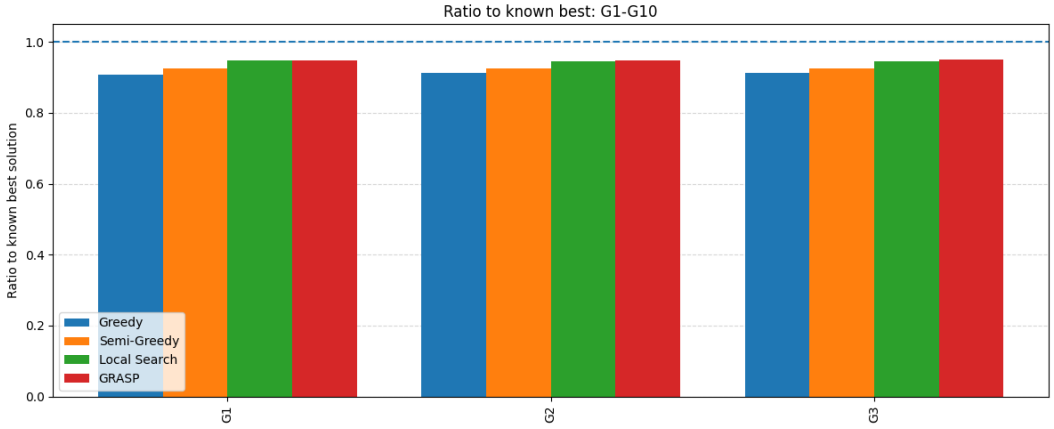

Figure 3: Ratio of each algorithm's cut weight to the known best solution, graphs G1–G3.

5 Comparison of Algorithms

5.1 Sample Detailed Comparison

Before averaging across all graphs with a known best solution, it is useful to inspect a small sample of instances in full detail. Table 2 lists raw cut-weight values for five representative graphs (two of the larger 2000-vertex instances and three of the 800-vertex instances), and Table 3 rescales the same values against each graph's known best solution, so that 1.0 corresponds to matching it exactly.

Graph	Known Best	Random	Greedy	Semi-Gr.	Local Srch	GRASP best	GRASP avg
G24	14,131	9,975.9	12,367.0	12,699.3	12,858.0	13,008.0	12,915.9
G37	8,009	5,887.1	7,264.0	7,368.8	7,376.0	7,476.0	7,445.0
G1	12,078	9,587.7	10,956.0	11,161.6	11,367.0	11,446.0	11,377.3
G36	7,996	5,893.2	7,225.0	7,355.3	7,354.0	7,456.0	7,433.7
G3	12,077	9,585.1	11,012.0	11,167.7	11,402.0	11,481.0	11,374.3

Table 2: Raw cut-weight values for a sample of five benchmark graphs.

Graph	Known Best	Random	Greedy	Semi-Gr.	Local Srch	GRASP best	GRASP avg
G24	14,131	0.706	0.875	0.899	0.910	0.921	0.914
G37	8,009	0.735	0.907	0.920	0.921	0.933	0.930
G1	12,078	0.794	0.907	0.924	0.941	0.948	0.942
G36	7,996	0.737	0.904	0.920	0.920	0.932	0.930
G3	12,077	0.794	0.912	0.925	0.944	0.951	0.942

Table 3: Same five graphs, scaled to known best (1.0 = matches known best).

Even on just this small sample, the same broad ordering that holds across the full data set is already visible: Random trails far behind, Greedy and Semi-Greedy close most of the remaining gap, and GRASP sits closest to the known best throughout. Local Search, now starting from a randomized solution rather than a semi-greedy one, lands noticeably closer to Semi-Greedy than to GRASP on these five graphs – for instance on G36 its scaled value (0.920) matches Semi-Greedy almost exactly, since without a semi-greedy head start, local search's improving moves have a weaker solution to refine.

5.2 Aggregate Results Across All Graphs with a Known Best

Averaged over the 24 graphs with a known best solution, the mean ratio to that known best is:

Algorithm	Mean ratio to known best
Randomized-1	0.525
Simple Greedy	0.851
Semi-Greedy-1 ($\alpha = 0.5$)	0.879
Local Search (average)	0.845
Local Search (best)	0.856
GRASP (average)	0.893
GRASP (best)	**0.904**

A few clear patterns emerge from the full 54-graph data set:

- **Randomized-1 is a weak baseline.** Reaching only about half of the known best on average (and even producing negative cut weights on the sparser graphs, since a purely random cut can perform worse than doing nothing when edge weights are mixed), it confirms that weight-aware construction is essential.
- **Greediness alone already closes most of the gap**, jumping to roughly 85% of the known best with a single deterministic pass.
- **Semi-greedy construction outperforms plain greedy on every graph**, by about 3.9% on average, since injecting randomness into an already-good greedy rule lets GRASP explore multiple promising starting points instead of being locked into one.
- **Local search starting from a random solution is noticeably weaker than semi-greedy construction alone**, falling behind Semi-Greedy on 14 of the 54 graphs despite refining its starting point with improving moves. Its average ratio to known best (0.845–0.856) is actually below Semi-Greedy's (0.879), which shows that $\delta(v)$-based local search can only improve on the quality of the solution it starts from – a strong starting point (as GRASP provides internally) matters more than the refinement step alone.
- **GRASP is by far the best-performing algorithm overall**, achieving the highest cut weight (best value) on 51 of the 54 graphs, with an average gain of about 4.0% over a single local-search run and never falling more than a fraction of a percent behind it on the 3 remaining graphs. GRASP's advantage comes from repeating the semi-greedy + local search cycle 50 times and keeping the best outcome, which both starts from a better construction and lets it escape the specific local optimum that a single random-start local-search run might get stuck in.
- **GRASP best and average values are very close**, as also seen in Figure 1, indicating the local-search phase reliably pulls different semi-greedy starts into a similar quality range rather than leaving high variance across iterations.

6 Conclusion

Across all 54 benchmark graphs, the results consistently support the theoretical motivation behind GRASP: each layer of the pipeline – from a random baseline, to greedy, to greedy-with-randomization, to the full construct-and-improve cycle repeated over many iterations – adds a measurable improvement in solution quality. GRASP is the strongest performer by a clear margin, reaching about 90.4% of the known best solution on average and outperforming a single local-search run on 51 of 54 instances, with an average gain of about 4%. Notably, the standalone Local Search algorithm – now starting from a randomized rather than a semi-greedy solution – performs *worse* on average than Semi-Greedy construction alone (0.85 vs. 0.88 of the known best), which highlights that the quality of local search's output depends heavily on the quality of its starting solution: refining a weak random cut with improving moves does not fully make up for skipping a good constructive heuristic. This is precisely why GRASP pairs semi-greedy construction with local search internally rather than relying on either alone, and it is also why GRASP's advantage over a standalone local-search run is now substantially larger than when local search started from a semi-greedy solution. In practice, the right algorithm choice still depends on the constraint at hand: GRASP's 50-iteration construct-and-improve cycle is far more expensive than a single greedy or semi-greedy pass, but for applications

where MAX-CUT quality matters more than running time, GRASP is the clear recommendation. Where a fast, reasonably good solution is sufficient, semi-greedy construction alone already captures most of the achievable improvement over a naive random cut, and outperforms local search run from a random start.